UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
ESTRUCTURA DE DATOS

# Report

Mauricio Joel Gómez Barrios - 202031478

03 de mayo de 2026

# Índice.

**Diseño de las estructuras** ..... **3**
- **1. Árbol AVL - ArbolAVL.java** ..... **3**
- **2. Árbol B - ArbolB.java** ..... **4**
- **3. Árbol B+ - ArbolBMas.java** ..... **5**
- **4. Tabla Hash - TablaHash.java** ..... **6**
- **5. Grafo - Grafo.java** ..... **6**
- **6. Cola y Pila — Cola.java / Pila.java** ..... **7**
- **7. GestionInventario - GestionInventario.java** ..... **7**

**Justificación de las Estructuras Implementadas** ..... **8**

**Análisis de Complejidad (Big-O)** ..... **9**
- 1. Árbol AVL (clave: nombre del producto) ..... 9
- 2. Árbol B (clave: fecha de caducidad) ..... 9
- 3. Árbol B+ (clave: categoría) ..... 10
- 4. Tabla Hash (clave: código de barras) ..... 10
- 5. Grafo y Dijkstra (Red de sucursales) ..... 10
- 6. Cola y Pila (TDAs Nativos) ..... 11
- 7. Operación Global: Agregar / Eliminar Producto (GestionInventario.java) ..... 11

**Comparación entre estructuras** ..... **12**

**Conclusiones** ..... **13**

**Diseño de las estructuras.**

El sistema utiliza siete estructuras de datos implementadas desde cero en Java, cada una especializada en un tipo de consulta sobre el catálogo de productos de cada sucursal. Todas comparten el mismo objeto Producto en memoria, no se duplican datos, solo se almacenan referencias. Cada inserción o eliminación se propaga a las estructuras del inventario de la sucursal correspondiente y si alguna falla, se ejecuta un rollback.

1. **Árbol AVL - ArbolAVL.java**
   Árbol binario de búsqueda con autobalanceo. La clave de ordenamiento es el nombre del producto. Garantiza altura O(log n) mediante cuatro rotaciones (LL, RR, LR, RL). Se emplea para búsqueda por nombre en tiempo logarítmico, listado alfabético en orden (in-order) y como fuente de nombres para la comparación de rendimiento.

| Atributo | Tipo | Descripción |
|---|---|---|
| producto | Producto | Puntero al objeto producto compartido |
| izquierdo | NodoAVL | Subárbol izquierdo (nombres menores) |
| derecho | NodoAVL | Subárbol derecho (nombres mayores) |
| altura | int | Altura del nodo, se actualiza tras cada rotación |

| Operación | Tipo | Descripción |
|---|---|---|
| insertar(p) | O(log n) | Inserta por nombre, ignora duplicados de nombre |
| buscarPorNombre(s) | O(log n) | Búsqueda binaria recursiva comparando String |
| eliminar(nombre) | O(log n) | Elimina y rebalancea, sucesor in-order si dos hijos |
| listarEnOrden() | O(n) | Recorrido in-order que devuelve lista ordenada |
| obtenerListaEnOrden() | O(n) | Retorna una lista in-order con todos los productos |
| crearGrafico(arch) | O(n) | Genera archivo .dot para Graphviz |

2. **Árbol B - ArbolB.java**

Árbol B de grado por defecto d = 3. La clave de ordenamiento es fecha_caducidad en formato YYYY-MM-DD. Como múltiples productos pueden compartir la misma fecha, cada clave apunta a una lista interna (List<Producto>) que agrupa todos los productos de esa fecha. Permite consultas eficientes por rango de fechas.

| Atributo | Tipo | Descripción |
|---|---|---|
| d | int | Grado mínimo d del árbol (default 3) |
| hoja | boolean | True si el nodo no tiene hijos |
| fechas | List | Arreglo de claves fecha YYYY-MM-DD |
| listasProductos | List | Listas de productos por clave fecha |
| hijos | List | Punteros a hijos (vacío si hoja) |

| Operación | Tipo | Descripción |
|---|---|---|
| insertar(p) | O(log n) | Si la fecha ya existe, agrega el producto a la lista de colisiones interna, sino se inserta la nueva clave realizando división de nodos al descender |
| eliminar(p) | O(log n) | Quita el producto de la lista interna. Si la lista queda vacía, elimina la clave (fecha) y rebalancea el árbol fusionando nodos (unir) o tomando claves prestadas de nodos hermanos (tomarDeAnterior/Siguiente) si no se cumple el grado mínimo |
| mostrarEnRango(fi,ff) | O(log n+k) | Navega al nodo más cercano a fechaInicio y recorre el árbol in-order hasta fechaFin. (k = cantidad de productos dentro del rango) |
| obtenerTodosLosProductos() | O(n) | Recorre todo el árbol de forma recursiva para devolver una lista plana con todos los productos ordenados por fecha de expiración. |
| crearGrafico(nombreArchivo) | O(n) | Genera el código .dot para Graphviz, mostrando las fechas y los nombres de los productos contenidos en cada nodo del árbol. |

3. **Árbol B+ - ArbolBMas.java**

Árbol B+ de grado por defecto d = 3. Los datos reales (Producto) se almacenan únicamente en los nodos hoja, los nodos internos solo contienen claves de enrutamiento. La clave de ordenamiento es la categoría. Como varias entradas pueden pertenecer a la misma categoría, cada hoja contiene listas de productos. Las hojas están enlazadas entre sí mediante punteros siguientes, lo que permite recorridos eficientes a nivel de hojas.

| Atributo | Tipo | Descripción |
|---|---|---|
| d | int | Grado mínimo d del árbol (default 3) |
| esHoja | boolean | Distingue un nodo hoja de un nodo interno |
| categorias | List<String> | Claves de categoría en el nodo (de enrutamiento o reales) |
| listasProductos | List<List<Producto>> | Listas de productos agrupados por categoría (solo en hojas) |
| hijos | List<NodoBMas> | Punteros a hijos (solo en nodos internos) |
| siguiente | NodoBMas | Enlace al siguiente nodo hoja (para recorridos lineales) |
| padre | NodoBMas | Puntero al padre para facilitar el rebalanceo e inserción durante división |

| Operación | Tipo | Descripción |
|---|---|---|
| insertar(p) | O(log n) | Navega hasta la hoja con buscarHoja(). Si la categoría existe, agrega el producto a su lista; si no, inserta una nueva clave en la hoja y llama a dividirNodo() si excede el límite (2d), propagando la inserción hacia arriba (padre) |
| buscarPorCategoria(cat) | O(log n+k) | Navega en tiempo logarítmico a la hoja correspondiente y extrae la lista de productos asociados. (k = productos en la categoría) |
| eliminarProducto(cat, cod) | O(log n) | Navega al nodo más cercano a fechaInicio y recorre el árbol in-order hasta fechaFin. (k = cantidad de productos dentro del rango) |
| obtenerTodosLosProductos() | O(n) | Recorre todo el árbol de forma recursiva para devolver una lista plana con todos los productos ordenados por fecha de expiración. |
| crearGrafico(arch) | O(n) | Genera el código .dot para Graphviz, mostrando las fechas y los nombres de los productos contenidos en cada nodo del árbol. |

4. **Tabla Hash - TablaHash.java**
   Tabla hash con encadenamiento separado usando listas enlazadas por cubo. La clave de búsqueda es el código de barras. La función hash procesa cada carácter del código con base 128 módulos capacidad. Cuando el factor de carga supera 0.80 se realiza un rehashing duplicando la capacidad (nuevaCapacidad = capacidad × 2 + 1). La búsqueda, inserción y eliminación son O(1).

| Operación | Tipo | Descripción |
|---|---|---|
| insertar(p) | O(1) | Calcula índice con funcionHash(), encadena al frente del cubo, rehash si factor de carga ≥ 0.80 |
| buscarPorCodigo(c) | O(1) | Calcula índice y recorre la cadena del cubo comparando código de barras |
| eliminar(cod) | O(1) | Localiza el nodo en el cubo y lo desenlaza |
| crearGrafico(arch) | O(n) | Genera .dot con la tabla completa y sus cadenas de colisión |

5. **Grafo - Grafo.java**
   Grafo dirigido y ponderado que modela la red de sucursales. Cada vértice es un objeto Sucursal y cada arista es un objeto Arista con peso tiempo y peso costo. El algoritmo de Dijkstra se implementa con arreglos ($O(V^2)$) para encontrar la ruta más corta entre dos sucursales, eligiendo el criterio de optimización (tiempo o costo) en tiempo de ejecución

| Operación | Tipo | Descripción |
|---|---|---|
| agregarSucursal(s) | O(1) | Añade la sucursal a la lista de adyacencia |
| agregarArista(orig,dest,t,c) | O(grado) | Añade o actualiza arista dirigida, evita duplicados |
| encontrarRutaMasCorta(o,d,t) | $O(V^2)$ | Dijkstra con arreglo, devuelve Pila con la ruta |
| realizarTransferencia(...) | $O(V^2+k)$ | Calcula ruta, procesa colas en cada sucursal intermedia |
| iniciarTransferenciaPasoAPaso | $O(V^2)$ | Crea objeto EstadoTransferencia y lo registra como activo |
| avanzarPasoTransferencia() | O(activos) | Avanza un paso cada transferencia activa en curso |
| eliminarSucursal(id) | O(V+E) | Elimina sucursal y todas las aristas que apunten a ella |
| cargarCSV(Sucursales, Aristas, Prod) | O(L*O) | Analizador léxico propio para leer archivos CSV respetando comillas (" "). Recorre las L líneas e inserta en la estructura correspondiente O. |

6. **Cola y Pila — Cola.java / Pila.java**
   Ambas estructuras se implementan mediante un TAD propio con nodos enlazados genéricos (NodoSimple<T>), la Cola (FIFO) administra las áreas de colaIngreso, colaTraspaso y colaSalida de cada sucursal. La Pila (LIFO) invierte las rutas calculadas por Dijkstra y gestiona el historial de devoluciones del inventario.

| Estructura | Operación | Tipo | Descripción |
|---|---|---|---|
| Cola | encolar(dato) | O(1) | Crea un nodo y lo enlaza al puntero finalCola |
| Cola | desencolar() | O(1) | Extrae y retorna el elemento del puntero frente, actualizando las referencias |
| Cola | obtenerElementos() | O(n) | Recorre la estructura entera para devolver una copia en lista (utilizado exclusivamente para la interfaz de visualización Graphviz) |
| Pila | push(dato) | O(1) | Crea un nodo y lo sitúa en la cima, apuntando al nodo anterior |
| Pila | pop() | O(1) | Extrae la cima y actualiza la referencia al nodo inferior |
| Pila | peek() | O(1) | Devuelve el valor de la cima sin modificar la estructura |

7. **GestionInventario - GestionInventario.java**
   Encapsula y coordina las seis estructuras internas de cada sucursal (ListaEnlazada no ordenada, ListaEnlazada ordenada, ArbolAVL, ArbolB, ArbolBMas y TablaHash). El frontend de la aplicación nunca interactúa directamente con las estructuras de datos, toda operación pasa a través de esta clase, garantizando la consistencia y atomicidad de los datos del inventario mediante la ejecución de un rollback si alguna estructura falla durante una inserción o eliminación múltiple.

| Atributo | Tipo | Descripción |
|---|---|---|
| listaNoOrdenada | ListaEnlazada | Registro sin orden, usado como base para la búsqueda secuencial por nombre |
| listaOrdenada | ListaEnlazada | Ordenada alfabéticamente por nombre, mantenida exclusivamente para las pruebas de rendimiento comparativas |
| avlNombres | ArbolAVL | Clave de ordenamiento: nombre. Búsqueda binaria en O(logn) y generación de listados alfabéticos balanceados |
| arbolFechas | ArbolB | Clave de ordenamiento: fecha_caducidad. Agrupación en bloques para consultas de productos próximos a vencer por rango de fechas |

| | | |
|---|---|---|
| arbolCategoria | ArbolBMas | Clave de ordenamiento: categoría. Recorrido de hojas enlazadas por categoría en tiempo O(logn+k) |
| tablaHash | TablaHash | Clave: código de barras. Lectura e inserción directa en O(1) para las operaciones de validación |
| pilaDevoluciones | Pila | Registro LIFO que almacena los objetos Producto que han sido devueltos a la sucursal y están a la espera de ser procesados o revisados |

**Justificación de las Estructuras Implementadas**

Cada estructura de datos se seleccionó e implementó para responder de forma óptima a un tipo específico de consulta o flujo sobre el inventario, maximizando la eficiencia en el caso de uso más frecuente para esa operación.

| Estructura | Caso de uso principal | Justificación de la elección |
|---|---|---|
| Árbol AVL | Búsqueda y listado por nombre | Garantiza un tiempo de O(log n) en todos los casos (mejor y peor) gracias al autobalanceo estricto con rotaciones. Supera ampliamente a una lista enlazada ordenada (O(n)) cuando el volumen de productos (n) crece |
| Árbol B | Consultas por rango de fechas de caducidad | Su grado d agrupa rangos, y su diseño maneja colisiones de la misma fecha mediante listas internas en cada clave. Tiempo óptimo O(log n+k) para rangos |
| Árbol B+ | Agrupación y listado por categoría | Separa las claves de enrutamiento de los datos reales. Los punteros secuenciales (siguiente) en el nivel de hojas permiten recorrer y extraer una categoría completa sin necesidad de retroceder a los nodos internos Útil para consultas tipo "Listar todos los productos de la categoría" |
| Tabla Hash | Búsqueda exacta por código de barras | Da acceso, inserción y búsqueda en O(1). Aunque el código de barras (clave) es único, las colisiones generadas por la operación módulo se resuelven mediante encadenamiento separado. El algoritmo de rehashing automático mantiene el factor de carga estrictamente por debajo de 0.80, garantizando cadenas cortas |
| Grafo y Dijkstra | Transferencias entre sucursales | Modela la red de distribución conectando sucursales (vértices) con rutas (aristas) mediante pesos duales (tiempo/costo). El uso de Dijkstra garantiza matemáticamente |

| | | |
|---|---|---|
| | | la ruta más barata o rápida según la elección del usuario. La inclusión de hilos permite simular el costo de tiempo real del recorrido |
| Cola (FIFO) | Flujo de productos en sucursales | El proceso físico de una bodega: los productos se procesan en su orden de llegada para las áreas de ingreso, traspaso y salida. Ofrece operaciones estructurales O(1) para encolar y desencolar |
| Pila (LIFO) | Devoluciones y reconstrucción de rutas | El registro de devoluciones espera una revisión, por lo que el último producto devuelto suele ser el primero a mano para ser procesado (LIFO). La pila permite invertir eficientemente en O(n) la ruta arrojada por el algoritmo de Dijkstra antes de presentarla al usuario |

**Análisis de Complejidad (Big-O)**

1. **Árbol AVL (clave: nombre del producto)**
   El AVL garantiza que la altura del árbol sea siempre h = O(log n) mediante rotaciones automáticas simples y dobles tras cada inserción o eliminación. Esto hace que la búsqueda, inserción y eliminación operen en O(log n) en el peor de los casos, no solo en el promedio. El recorrido in-order visita los n nodos exactamente una vez: O(n).

| Operación | Tipo | Caso |
|---|---|---|
| Búsqueda por nombre | O(log n) | Siempre (garantizado por el balanceo) |
| Inserción y rebalanceo | O(log n) | Siempre |
| Eliminación y rebalanceo | O(log n) | Siempre |
| Recorrido in-order | O(n) | Visita y extrae todos los elementos ordenados |

2. **Árbol B (clave: fecha de caducidad)**
   Con grado d, la altura máxima es O(log n). Las operaciones de búsqueda interna, inserción y eliminación navegan desde la raíz hasta una hoja realizando comparaciones en O(log n). La búsqueda por rango agrega un costo lineal proporcional a los k productos que caen dentro de dicho rango.

| Operación | Tipo | Caso |
|---|---|---|
| Búsqueda (interna) | O(log n) | Siempre |
| Inserción y división | O(log n) | Siempre |
| Eliminación y rebalanceo | O(log n) | Siempre (incluye fusiones o préstamos) |

| | | |
|---|---|---|
| Búsqueda en rango de fechas | O(log n + k) | O(log n + k) k = cantidad de productos devueltos en el rango |
| Extracción total de productos | O(n) | Recorrido completo del árbol |

3. **Árbol B+ (clave: categoría)**
   La estructura encierra los datos en las hojas. La inserción y eliminación son O(log n) siguiendo la ruta raíz-hoja. La búsqueda por categoría opera en O(log n + k): invierte O(log n) para localizar la primera hoja que contiene la categoría, y O(k) para recolectar los k productos iterando linealmente a través de los punteros de enlace (siguiente) entre las hojas.

| Operación | Tipo | Caso |
|---|---|---|
| Inserción y división | O(log n) | Siempre |
| Eliminación de producto | O(log n) | Siempre |
| Búsqueda/Recorrido por categoría | O(log n + k) | k= cantidad de productos extraídos de la categoría |
| Extracción total de productos | O(n) | Desciende a la primera hoja y recorre la lista enlazada base |

4. **Tabla Hash (clave: código de barras)**
   La función hash distribuye uniformemente los códigos en los cubos. El rehashing dinámico mantiene el factor de carga estrictamente por debajo de 0.80, garantizando cadenas de colisión muy cortas y proveyendo un rendimiento constante O(1) para sus operaciones base.

| Operación | Tipo | Caso |
|---|---|---|
| Búsqueda exacta | O(1) | Factor de carga controlado < 0.80 |
| Inserción | O(1) | Contempla la penalización ocasional del rehash |
| Eliminación | O(1) | Siempre |
| Rehash (Aumentar tamaño) | O(n) | Solo se ejecuta cuando el factor de carga supera el límite |

5. **Grafo y Dijkstra (Red de sucursales)**
   El algoritmo de Dijkstra se implementó utilizando arreglos paralelos simples, lo que arroja una complejidad de O(V^2), donde V es el número de sucursales. Dado que una red de sucursales de supermercado suele ser un grafo pequeño y denso (V normalmente < 100), esta implementación directa es adecuada y evita el exceso de memoria.

| Operación | Tipo | Caso |
|---|---|---|
| Dijkstra (ruta más corta) | O(V^2) | Arreglos lineales para distancias y visitados, óptimo para V < 1000 |

| | | |
|---|---|---|
| Agregar sucursal | O(1) | Inserción al final de la lista de vértices |
| Agregar arista | O(V+grado) | Requiere búsqueda lineal de vértices (O(V)) y validación de duplicados |
| Transferencia completa | O(V^2+R) | Calcula la ruta O(V^2) más R pasos de procesamiento en colas a lo largo del trayecto. |

6. **Cola y Pila (TDAs Nativos)**

| Estructura | Operación | Tipo | Caso |
|---|---|---|---|
| Cola | encolar / desencolar | O(1) | Mantenimiento de punteros aislados (frente y finalCola), no hay recorrido ni corrimiento |
| Cola | obtenerElementos() | O(1) | Recorrido exhaustivo copiando todos los nodos para alimentar la vista gráfica |
| Pila | push / pop / peek | O(1) | Operaciones limitadas exclusivamente a un puntero único (cima) |

7. **Operación Global: Agregar / Eliminar Producto (GestionInventario.java)**
   Cada inserción o eliminación individual solicitada por el usuario o leída desde el CSV se propaga atómicamente a las seis estructuras internas del inventario de la sucursal implicada. Por lo tanto, el tiempo de procesamiento global de una inserción está definido por la suma de los tiempos de todas las estructuras.

| Estructura | Operación de Inserción | Complejidad (Peor Caso) |
|---|---|---|
| Lista No Ordenada | insertar | O(n) (verifica duplicidad secuencialmente antes de insertar al final) |
| Lista Ordenada | insertar | O(n) (búsqueda lineal de la posición correcta) |
| Árbol AVL | insertar | O(log n) |
| Árbol B | insertar | O(log n) |
| Árbol B+ | insertar | O(log n) |
| Tabla Hash | insertar | O(1) |
| Global (Costo Total) | agregar / eliminar | O(n) (dominado por las Listas Enlazadas) |

**Comparación entre estructuras**

La siguiente tabla resume las características clave de cada estructura, permitiendo identificar cuándo usar cada una según el tipo de consulta y el tamaño del conjunto de datos.

| Estructura | Búsqueda | Inserción | Eliminación | Rango | Orden | Caso de uso ideal |
|---|---|---|---|---|---|---|
| Lista no ord. | O(n) | O(1) | O(n) | No | No | Registro maestro, inserción masiva rápida |
| Lista ordenada | O(n) | O(1) | O(n) | Sí | Sí | Comparativas, listado lineal |
| Árbol AVL | O(log n) | O(log n) | O(log n) | No | Sí | Búsqueda por nombre, listado alfabético |
| Árbol B | O(log n) | O(log n) | O(log n) | Sí | Sí | Filtros por rango de fechas |
| Árbol B+ | O(log n) | O(log n) | O(log n) | Sí | Sí | Agrupación por categoría, hojas enlazadas |
| Tabla Hash | O(1) | O(1) | O(1) | No | No | Búsqueda exacta por código de barras |
| Grafo y Dijkstra | $O(V^2)$ | O(1) | O(V+E) | N/A | N/A | Ruta óptima entre sucursales |

## Conclusiones

1) El proyecto demostró que elegir la estructura de datos correcta transforma la velocidad del sistema. El árbol AVL resultó ser más rápido que la lista enlazada para búsquedas por nombre con +1,000 productos, y esta ventaja crece logarítmicamente a medida que el catálogo aumenta. La clave fue asignar la herramienta óptima para cada tarea: listas simples para inserción rápida, AVL para búsqueda por nombre, Árbol B para filtros de fechas, Árbol B+ para categorías y Tabla Hash para acceso O(1) por código.

2) Implementar las estructuras desde cero en Java, permitió comprender el funcionamiento interno real de cada estructura: cómo se resuelven las colisiones en la tabla hash, por qué las rotaciones del AVL preservan el orden y cómo las hojas enlazadas del B+ hacen eficiente el recorrido por categoría.

3) GestionInventario mantiene el código del frontend completamente separado de las estructuras internas. Esto permite reemplazar, por ejemplo la tabla hash por otra implementación sin tocar ninguna línea de la interfaz gráfica. La decisión de que solo la lista no ordenada sea dueña de la memoria del objeto Producto evita dobles liberaciones y simplifica la gestión del ciclo de vida de los datos.

4) El proyecto demostró que no existe una estructura perfecta para todo. Cada una brilla en su especialidad: listas para inserción rápida, AVL para búsqueda puntual, B para rangos de fechas, B+ para categorías completas, Hash para acceso O(1) por código único, Grafo para rutas entre nodos y Cola/Pila para modelar flujos y estados temporales. Un sistema bien diseñado combina todas estas herramientas y acepta el costo de mantenerlas coordinadas a cambio de responder a la máxima velocidad cualquier tipo de consulta del usuario.